VALUE ADDED PROBLEM 2:

Problem 2: Given a string s, composed of different combinations of '(' , ')', '{', '}', '[', ']'. Determine whether the Expression is balanced or not. An expression is balanced if: 1. Each opening bracket has a corresponding closing bracket of the same type. 2. Opening brackets must be closed in the correct order. Concept: Stack

**Examples :**

**Input:** s = "[{()}]"
**Output:** true
**Explanation:** All the brackets are well-formed.

**Input:** s = "[()()]{}"
**Output:** true
**Explanation:** All the brackets are well-formed.

**Input:** s = "([]"
**Output:** false
**Explanation:** The expression is not balanced as there is a missing ')' at the end.

**Input:** s = "({[]})"
**Output:** false
**Explanation:** The expression is not balanced as there is a closing ']' before the closing '}'.

```
#include <stdio.h>

#include <string.h>

#include <stdbool.h>

#define MAX 1000000

char stack[MAX];

int top = -1;

void push(char c) {

   stack[++top] = c;}

char pop() {

   if (top == -1) return '\0';

   return stack[top--];}

bool isEmpty() {

   return top == -1;}

bool isMatch(char open, char close) {

   return (open == '(' && close == ')') ||

         (open == '{' && close == '}') ||

         (open == '[' && close == ']');
```

```
}
bool isBalanced(char* s) {
   top = -1;
for (int i = 0; s[i] != '\0'; i++) {
      char ch = s[i];
 if (ch == '(' || ch == '{' || ch == '[') {
          push(ch)    }  else if (ch == ')' || ch == '}' || ch == ']') {
          if (isEmpty()) return false;
          char top_char = pop();
          if (!isMatch(top_char, ch)) return false;  }
   }
  return isEmpty();
}
int main() {
  printf("ARYAN SHEGOKAR , 25070521180 , SECTION C(2)\n\n");
char* tests[] = {"[{()}]", "[()(){}", "([)", "[{()})"};
   int n = 4;
 for (int i = 0; i < n; i++) {
      printf("Input:  \"%s\"\n", tests[i]);
      printf("Output: %s\n\n", isBalanced(tests[i]) ? "true" : "false");
  }


   return 0;
```

```
Output
ARYAN SHEGOKAR , 25070521180 , SECTION C(2)

Input:  "[{()}]"
Output: true

Input:  "[()(){}"
Output: false

Input:  "([)"
Output: false

Input:  "[{()})"
Output: false


=== Code Execution Successful ===
```

}